DEVELOPER RE-ENTRY MANUAL

PDF Workspace: Code Breakdown and Maintenance Guide

This document explains how the codebase is organized, how the major modules work together, and which invariants matter when extending the system. It is intended both as a code breakdown for external review and as an internal manual for resuming work without destabilizing key learning flows.

What this app is in code terms

The repository is a full-stack research-reading system. The frontend is a React workspace shell with a PDF reader, HUD panels, and full-page study workspaces. The backend is a FastAPI service with persistent models for PDFs, highlights, study cards, review logs, and research sessions. ChromaDB stores retrieval chunks, S3 stores source files, and Claude powers reasoning-heavy operations.

How to think about the system

The safest mental model is a pipeline:

```
PDF upload
→ chunking + embeddings + storage
→ reading and selection
→ chat / save-to-index
→ highlight entry + QA cards
→ review queue + grading + FSRS
→ research session organization
→ comparative analysis
```

SHELL

Top bar controls workspace mode. Reader keeps HUD. Index, Review, and Compare mount full-page workspaces.

STATE

Zustand is the application spine. It holds selected PDF, sessions, highlight index, chat state, and review mode.

PERSISTENCE

Postgres is the source of truth for PDFs, highlights, QAs, sessions, and review logs.

RETRIEVAL

ChromaDB anchors user interactions back to chunk-level source evidence.

STUDY

Review logic uses LLM grading plus FSRS. This is one of the most breakage-sensitive areas.

ANALYSIS

Comparative analysis sits above sessions and depends on both PDFs and user-generated artifacts.

1. APPLICATION SHELL AND FRONTEND COMPOSITION

How the UI is assembled

The top-level frontend entry is `frontend/src/App.jsx`. This file defines the shell contract for the app: there are four workspace modes, `reader`, `index`, `compare`, and `review`. The reader mode is the default interactive mode and mounts three major columns: `PDFSidebar`, `PDFViewer`, and `ChatPanel`.

File	Role in the larger scheme
`App.jsx`	Defines workspace shell, top navigation tabs, and the split between Reader mode and full-page workspaces.
`PDFSidebar.jsx`	Left navigation and session organizer. Owns upload entry point, session management modal flow, and session-first PDF selection.
`PDFViewer.jsx`	Main document surface. Connects page rendering, TOC navigation, highlight overlays, and the jump back from index/review to source.
`ChatPanel.jsx`	Reader-side conversational assistant. Also acts as an interaction bridge into indexing and comparison.
`WorkspaceIndex.jsx`	Full-page index workspace using session and PDF navigation, currently reusing the same underlying index data as the HUD.
`WorkspaceReview.jsx`	Full-page review queue organizer for sessions and papers, while the actual recall/grading flow still uses the review overlay.
`WorkspaceCompare.jsx`	Full-page comparative analysis workspace driven by session-level analysis data.
`ReviewSession.jsx`	Dedicated modal/full-screen review flow that runs during active study.

Important composition rule

The HUD and the full-page workspaces are related but not identical. The HUD is for in-reading interaction. The full-page tabs are intended to become more structured deep-dive views. When modifying them, do not assume the panel and full-page versions should always share identical presentation logic.

Do not break this:

`reader` mode is the integration point where PDF viewing, side HUD, and session context all coexist. A change that works in a full-page workspace but breaks the tri-pane reader layout will usually damage the actual user flow more than it appears.

2. ZUSTAND STORE AND FRONTEND STATE CONTRACT

The store is the frontend spine

`frontend/src/store/index.js` is one of the most important files in the codebase. It is where frontend state, backend hydration, UI mode switches, and data normalization come together. If a new feature needs selected PDF context, highlight index entries, chat history, or review launch state, it almost certainly touches this file.

Main responsibilities

- Store the selected PDF and active workspace mode.
- Hydrate highlights from the database on PDF selection.
- Keep chat history per PDF through `chatHistoriesByPdf`.
- Persist lightweight UI preferences such as workspace mode and placement suggestion toggle.
- Normalize backend snake_case responses into camelCase UI objects.
- Provide mutation helpers for stars, flags, notes, synthesis, and Q&A actions.

Key state slices

- `researchSessions`, `pdfs`, `selectedPdf`
- `highlightIndex`
- `chatHistory` and `chatHistoriesByPdf`
- `reviewMode` and `reviewScope`
- `indexFocus`, `navRequest`, `flashHighlight`
- `selectionContext` and reader-side temporary interaction state

Normalization is a maintenance boundary

The store exposes `normalizeEntry` and `normalizeQA`. These functions convert backend rows into frontend-friendly structures. This is a critical safety boundary because many components assume camelCase fields and consistent nested shapes. If a backend response changes and these normalizers are not updated, breakage will often appear far away from the actual change.

```
backend row → normalizeEntry / normalizeQA
→ highlightIndex array in store
→ HighlightIndex / PDFViewer / WorkspaceIndex / Review launch
→ downstream UI behavior
```

Why the store matters for not breaking the app

Many features that look independent actually share the same normalized objects. For example, a change to Q&A shape affects the index, the review launcher, full-page workspaces, and compare evidence summaries. When in doubt, change the adapter first and keep components dumb.

High-risk mistake:

Writing component-local shape fixes instead of updating the store normalizers. That creates inconsistent assumptions across the app and is one of the fastest ways to break index, review, and compare together.

3. READER, CHAT, AND INDEX CAPTURE PATH

How document interaction turns into stored knowledge

The reading and capture path is centered around `PDFViewer.jsx`, `ChatPanel.jsx`, and the highlights API. The code is designed so that a user can either interact directly with a highlighted passage or ask a question in chat and still end up with a reusable index entry and study card.

ChatPanel responsibilities

- Build chat requests for the selected PDF, optionally with selected text and section title.
- Choose between quick mode and full structured response mode.
- Render grounded answers with page citations that can jump back into the PDF.
- Create save candidates that can become index entries and study cards.
- Extend partial selections toward sentence boundaries when source chunk text is available.
- Handle dedup conflicts when logging study cards into the index.

Backend side of chat

`backend/app/routers/chat.py` is thin by design. It validates the document, passes the request to `chat_service.chat()`, and returns answer plus sources. Additional helper endpoints exist for concept extraction, synthesis generation, and study-card preparation.

`chat_service.py` is where the heavier behavior lives: response templates, retrieval routing, document-vs-web fallback logic, and session-aware framing.

Highlight and QA persistence

The frontend `saveToIndex` flow in the store either appends a new QA to an existing highlight entry or creates a new highlight entry first and then posts a QA under it. The backend router `backend/app/routers/highlights.py` routes all QA creation through `card_service.create_card()` so study-question canonicalization and dedup happen in one place.

```
reader selection or chat question
→ ChatPanel builds save candidate
→ store.saveToIndex()
→ POST /api/pdfs/{pdf_id}/highlights (if needed)
→ POST /api/highlights/{highlight_id}/qa
→ card_service canonicalizes + dedups
→ store re-fetches highlights
→ normalized entry appears in index and review surfaces
```

Do not break this:

The backend owns study-question canonicalization. If the frontend starts reintroducing custom parsing logic for question text, dedup quality and review consistency will diverge immediately.

Page 4 — Reader, chat, and indexing path

4. REVIEW SYSTEM, GRADING, AND SCHEDULING

The most breakage-sensitive learning loop

The review system is a thesis-critical part of the codebase. Its main frontend file is `frontend/src/components/ReviewSession.jsx`, and its backend route is `backend/app/routers/review.py`. This flow is intentionally designed to capture recall before grading is revealed.

Layer	Responsibility
`ReviewSession.jsx`	Implements the three-phase UX: recall, graded response, and done state. Controls reveal mode and confidence entry before submission.
`review.py`	Validates request, gets source text, runs grading, updates FSRS card state, writes review_log row, and returns grading feedback.
`grading_service.py`	Contains Claude grading prompt, score mapping, source retrieval for grading, and FSRS card reconstruction/persistence helpers.
`models/review.py`	Defines `ReviewLog`, prompt and rubric versions, session events, and human grading structures.

Critical logic

- User confidence is collected before grading is shown.
- Claude grades the recall on three dimensions: core claim, supporting detail, faithfulness.
- FSRS rating is derived from Claude's overall grade, not the user confidence score.
- Review logs preserve model version, prompt version, rubric version, and reveal telemetry.
- Due-card queries filter archived QAs and diversify nearby cards by section.

Why this area is fragile

The review flow has both UX and methodological constraints. If the order of operations changes, for example confidence being recorded after the grade, or full source text being shown by default, the resulting study logs become less defensible for research analysis. This is not just a UI preference.

```
fetch due cards
→ user sees study_question
→ confidence + recall_text entered
→ submit_review()
→ get_source_text() from chunk ids
→ Claude grading
→ compute overall score
→ map score to FSRS rating
→ persist updated qa_pair state
→ append review_log row
→ return grade + feedback + next schedule state
```

Do not break this:

Keep the one-call submission contract in `/api/review/submit`. Confidence, recall text, reveal metadata, grading, and scheduling all assume one transaction-like path. Splitting this casually would complicate both data integrity and thesis analysis.

5. RESEARCH SESSIONS AND COMPARATIVE ANALYSIS

Organization and cross-paper reasoning layers

The session layer and compare layer are the main architectural moves that elevate the project from single-paper assistance to research workspace. These are handled mostly by `backend/app/models/research_session.py`, `research_session_service.py`, `routers/research_sessions.py`, and `comparative_analysis_service.py`.

Research session code responsibilities

- Create and maintain default sessions such as Learning Design Research and Unsorted Research.
- Provide heuristics for session suggestion and PDF placement.
- Maintain many-to-many PDF membership while preserving a legacy primary-session pointer.
- Support create, edit, delete, assign, remove, suggest, and backfill flows.

Comparative analysis code responsibilities

- Build deterministic baseline rows from indexed evidence.
- Sample PDF chunks directly from ChromaDB as primary evidence.
- Generate per-paper AI analyses.
- Generate and normalize a session matrix with fixed and session-specific axes.
- Cache results on the research session and enrich weak AI output with deterministic fallbacks.

Important architectural decision

Comparative analysis is PDF-first, not index-first. Saved Q&A material is used as a secondary signal showing what the reader has already surfaced. This is an important boundary because otherwise the compare view would become a summary of interaction artifacts rather than a comparison of the actual documents.

```
session id
→ get session memberships
→ fetch PDFs in session
→ fetch saved highlight entries / QA anchors
→ sample PDF chunks from Chroma
→ generate per-paper analyses
→ synthesize matrix, insight cards, gaps, guidance
→ cache on research session
→ WorkspaceCompare renders full-page research desk
```

Do not break this:

Avoid mutating ORM relationships during read-only comparative analysis. This already caused one integrity failure previously. Filtering archived QAs must be done without rewriting relationship fields on live ORM objects.

6. BACKEND SERVICES, DATA MODEL, AND EXTERNAL DEPENDENCIES

Where domain logic actually lives

The backend is deliberately organized so routers stay relatively thin and the heavier logic sits inside services. This is useful both for maintenance and for thesis transparency because it makes the reasoning steps more inspectable.

Service / model	What it does in the larger scheme
`pdf_service.py`	Extracts text and chunks from uploaded PDFs.
`embedding_service.py`	Generates MiniLM embeddings for retrieval and similarity work.
`chroma_service.py`	Persists chunk vectors and supports passage retrieval / chunk resolution.
`s3_service.py`	Stores and serves original PDF files.
`toc_service.py`	Extracts TOC or approximates it via font heuristics.
`chat_service.py`	Owns answer generation strategy and response prompting.
`card_service.py`	Owns card creation contract, study-question derivation, and dedup.
`grading_service.py`	Owns rubric-based grading and FSRS coupling.
`ontology_service.py`	Generates paper ontology, rhetorical facets, topic tags, and recomputation flows.
`research_session_service.py`	Owns session defaults, placement heuristics, and session assignment rules.
`comparative_analysis_service.py`	Owns the session-level literature-review desk.

Core database entities

- `PDFDocument` links file metadata to ontology and session state.
- `HighlightEntry` is the parent storage unit for indexed passages.
- `QAPair` is both study artifact and spaced-repetition card.
- `ReviewLog` is the empirical ledger of actual review events.
- `ResearchSession` is the organization unit for multi-paper work.

External system dependencies

- **Postgres** for persistent structured state.
- **ChromaDB** for chunk retrieval and evidence anchoring.
- **S3** for file persistence.
- **Anthropic** for chat, grading, ontology, and compare generation.
- **SentenceTransformers** for embeddings and dedup similarity.

Maintenance rule:

When debugging behavior, check the router last. The router usually only validates shape and delegates. The actual logic almost always sits in a service or in the frontend store adapter.

7. SAFE RE-ENTRY, TESTING, AND BREAKAGE AVOIDANCE

How to jump back into work without destabilizing the app

Where to start when resuming work

- Start from the user flow, not from a file. Ask whether the change belongs to reading, indexing, review, sessions, or compare.
- Identify the boundary file: shell component, store adapter, router, or service.
- Inspect the corresponding Playwright coverage before editing.
- Prefer additive changes to data shape and serialization over replacing existing fields.

High-value regression areas

- Chat save-to-index flows and duplicate resolution.
- Review overlay sequencing and reveal telemetry.
- Session assignment and sidebar rendering.
- Comparative analysis cache refresh.
- Store normalization after backend field changes.

Useful existing tests

- `step2_dedup_verify.js`
- `step3_recall_first_verify.js`
- `workspace_shell_part1.js`
- `workspace_index_review_part2.js`
- `workspace_compare_part3.js`
- `part3_comparative_analysis.js`

Operational invariants

- Postgres is the source of truth for PDFs, highlights, QAs, sessions, and review logs.
- The frontend store is the source of truth for normalized frontend shapes.
- The backend owns study-question canonicalization and dedup decisions.
- The review pipeline must remain recall-first.
- Comparative analysis must remain PDF-first.
- Legacy artifacts should be preserved whenever possible rather than silently deleted or rewritten.

Recommended edit strategy

```
1. Reproduce current behavior with Playwright or manual path.
2. Identify whether the bug is shape, persistence, retrieval, or rendering.
3. If shape: update adapter / serializer first.
4. If behavior: update service first, router second, component last.
5. Re-run the narrow test.
6. Re-run at least one end-to-end workspace flow touching the same feature family.
```

Current active caution area:

The next major refactor target is contextual integrity in Index and Review. Questions, source passages, and study cards need stronger provenance and context modeling. Any implementation here should preserve existing chats, review logs, and index artifacts while improving how context is derived and displayed.